HDFC Bank Stock Analysis and Next-Day Price Prediction Using Machine Learning Regression Models

Y Sudheer Reddy
Dept. of Computer Science and Engineering
Lovely Professional University
Phagwara, Punjab, India
Registration No. 12407920

***Abstract*—This paper presents an end-to-end data science study of historical HDFC Bank stock data, covering data cleaning, exploratory analysis, and prediction of the next-day adjusted closing price. The dataset contains 6,567 daily observations from 1 January 1996 to 21 January 2022, of which 6,560 remain after cleaning. The exploratory analysis covers the long-term price trend, trading volume, daily returns, correlation among market variables, and year-end prices. Lag, return, moving-average, volatility, price-range, and calendar features are engineered, and the target is the adjusted close of the next available trading observation. Five regression models (Linear, Ridge, Random Forest, Gradient Boosting, and Extra Trees) are compared on a chronologically later test set using mean absolute error (MAE), root mean squared error (RMSE), and the coefficient of determination (R^2). Linear Regression performs best, with an MAE of 13.1117, an RMSE of 19.5296, and an R^2 of 0.9949, while the tree-based ensembles obtain negative R^2 values in this run. The study is educational and is not investment advice.**

Keywords—stock price prediction, exploratory data analysis, feature engineering, regression, time series, scikit-learn

I. Introduction

Data science combines data collection, preparation, exploration, statistical reasoning, visualization, and predictive modeling to extract useful information from data [1]. Machine learning is the part of this process in which algorithms learn patterns from historical examples and use them to generate predictions. A complete project should not begin and end with model training: it should establish a clear problem, understand the source and limitations of the data, prepare the records carefully, investigate patterns, build appropriate features, compare models fairly, and communicate the findings with suitable visual evidence.

This paper studies historical market data for HDFC Bank, supplied as the file HDFCBANK.NS.csv, which contains date-wise opening, high, low, closing, and adjusted closing prices together with trading volume. These fields allow a study of how the price changes over time, how trading activity varies, how daily returns are distributed, and how price-related variables move together. All analysis was implemented in Python using pandas [2], NumPy [3], Matplotlib [4], and scikit-learn [5].

The machine learning component is framed as a regression problem. The target is the adjusted closing price in the next available trading-day row, and historical prices and derived indicators are the predictors. Five regression algorithms are compared, and the model with the lowest root mean squared error on a chronologically later test period is selected.

The study is educational. A historical stock-price model cannot guarantee future returns or provide a reliable trading instruction. Financial markets are influenced by company announcements, economic conditions, interest rates, policy decisions, global events, liquidity, and investor expectations, which are not necessarily represented in a price-and-volume file.

A. Background and Motivation

Financial datasets are useful for demonstrating time-series analysis because observations have a natural order. Unlike many ordinary tabular problems, a future observation must not be used to predict an earlier one. This makes chronological splitting, lag construction, and leakage prevention especially important. A random train-test split can place later records in the training set and earlier records in the test set, producing an evaluation that does not resemble the intended forecasting situation.

HDFC Bank is the subject of this case study because the supplied dataset is identified by the ticker-style file name HDFCBANK.NS.csv. The paper does not attempt to explain every movement in the share price. Instead, it demonstrates a repeatable pipeline using the fields available, and the motivation is practical experience with the decisions in an end-to-end workflow: handling missing values, detecting duplicates, calculating returns and moving averages, selecting a target, and comparing models with more than one metric.

B. Problem Statement

The problem is to analyze historical HDFC Bank market observations and investigate whether selected price, volume, rolling-statistic, and calendar features can estimate the next available trading day's adjusted closing price. The analysis must also describe the structure and quality of the dataset, identify patterns through visualizations, and compare multiple regression models on a time-ordered test set. The prediction is defined at the row level: for each date t, the target is the adjusted close of the next available record t+1. This is the next trading observation rather than strictly the next calendar day, because exchanges do not trade on every date. The error is measured in adjusted-price units.

C. Objectives

- Load, inspect, and clean the dataset by parsing dates, converting numeric fields, removing duplicates, and handling missing or invalid values.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

- Analyze the adjusted closing-price trend, daily trading volume, daily returns and their distribution, correlations among the market variables, and year-end prices.
- Engineer lag, return, moving-average, volatility, price-range, and calendar features.
- Train and compare five regression models using MAE, RMSE, and R^2 on a chronological test set.

D. Scope and Limitations

The scope is limited to the records in the supplied file. Financial statements, market indices, news, macroeconomic indicators, and sector information are not used, and the target is a numeric price level rather than a buy/sell recommendation. The data extend from 1 January 1996 to 21 January 2022, so the results describe a historical period and not current market conditions; a refreshed dataset would require all results to be regenerated.

The cleaning workflow interpolates or carries values for selected missing feature fields. This keeps the workflow consistent but may not be appropriate for every market-data gap, so the choice is disclosed here. The reported scores depend on the specific period, features, split point, and model settings.

II. Dataset and Preprocessing

A. Source and Description

The dataset is a structured daily time-series table in which each row is one date's market observation. The ".NS" suffix in the file name follows the ticker convention commonly used for National Stock Exchange listings, but the download provider cannot be established from the file name alone: [6]. The fields are summarized in Table I. Date provides the temporal ordering, Open, High, Low, and Close describe the reported price range and closing value, Volume is the reported number of shares traded, and Adj Close is the adjusted closing price used for the principal trend and as the prediction target.

TABLE I. Dataset Attributes

Attribute	Description	Role in project
Date	Observation date	Sort order, calendar features
Open	Opening price	Input feature
High	Highest reported price	Input and range feature
Low	Lowest reported price	Input and range feature
Close	Closing price	Input feature
Adj Close	Adjusted closing price	Trend analysis and target
Volume	Reported trading volume	Input and activity analysis

B. Data Cleaning Workflow

The workflow is designed to be transparent, and each operation has a reason: dates must be valid to sort the series, numeric conversion is required for arithmetic, duplicate removal avoids counting an observation twice, and the essential fields are required for the target definition. The steps were as follows.

- Load the CSV with pandas and inspect the first records, shape, and column names.
- Parse Date to datetime (invalid values coerced to missing) and sort the observations chronologically.
- Remove exact duplicate rows; none were found in the recorded run.
- Convert Open, High, Low, Close, Adj Close, and Volume to numeric values.
- Treat negative volume as invalid and set it to missing; treat High below Low as an invalid range for those fields.
- Drop rows lacking Date, Close, or Adj Close, then interpolate and forward/backward fill the remaining gaps in selected feature fields.
- Save the result as HDFCBANK_cleaned.csv.

C. Data Quality Summary

Table II summarizes the checks. The missing-cell count is taken from the recorded project execution and should be regenerated if the notebook is rerun on a changed copy of the source file.

TABLE II. Data Quality Checks

Check	Observed result
Initial row count	6,567
Cleaned row count	6,560
Missing cells before cleaning	42
Duplicate rows removed	0 in the recorded run
Date range	1996-01-01 to 2022-01-21

D. Source Verification and Reproducibility

A reliable report should allow another reader to locate the same data. The source citation should therefore state the provider, ticker symbol, download endpoint or historical-data page, date accessed, and any data terms. The original file is retained separately from the cleaned file because preprocessing changes the records; keeping both allows the cleaning steps to be reproduced, the row counts compared, and the treatment of missing or invalid values audited. For reproducibility, the notebook should be run from top to bottom in a clean Python environment with the package versions and source file retained, and the reported numbers should match its output.

III. Exploratory Data Analysis

Exploratory data analysis (EDA) examines a dataset before conclusions are drawn or models are fitted. It uses descriptive statistics, tabular inspection, and visualization to understand data types, ranges, missing values, unusual observations, distributions, and relationships, and to decide whether further cleaning or feature construction is needed. Here the EDA is organized around five objectives, each answered with a chart suited to its question: a line chart for date-indexed series, a histogram for a distribution, and a correlation heat map for pairwise linear association.

A. Descriptive Statistics

Table III summarizes central tendency, dispersion, and range for the numeric fields. Volume is reported in millions because it differs from the price fields in scale by several orders of magnitude.

TABLE III. DESCRIPTIVE STATISTICS OF THE CLEANED DATA (N = 6,560)

Variable	Mean	Std. dev.	Min	Max
Open	337.150	428.941	2.435	1,705.000
High	340.553	432.702	2.435	1,725.000
Low	333.371	424.684	2.395	1,671.000
Close	337.044	428.706	2.435	1,688.700
Adj Close	328.144	426.935	1.976	1,688.700
Volume (M)	4.942	5.712	0.000	201.130

The mean and median give different views of a typical value, and for Adj Close the median (122.550) is far below the mean (328.144), reflecting the long period of low prices before the later rise. The large standard deviations relative to the means show that the price level changed substantially over the sample.

B. Stock Price Trend

Fig. 1 plots the adjusted closing price against date. Over the sample the price ranges from a minimum of 1.976 to a maximum of 1,688.700. The chart shows a long period of low price levels followed by sustained growth and a sharp temporary decline in early 2020. A long date range can contain several market regimes, including gradual appreciation, correction, and rapid repricing. Because the vertical axis shows nominal levels, a fixed price change represents a different proportional move at different levels, so the chart is read together with the return analysis.

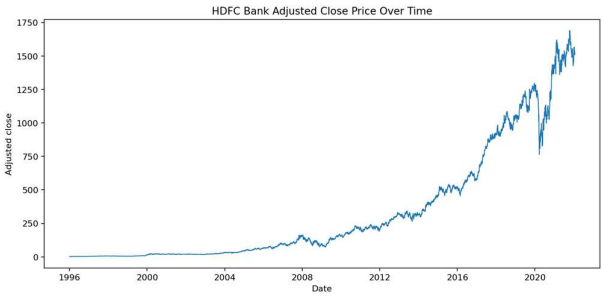


Fig. 1. HDFC Bank adjusted closing price over time.

The adjusted close is used rather than the raw close alone because adjusted series commonly account for certain corporate actions, depending on the provider's methodology, which was not independently verified. The chart is a historical record and does not identify causes; explaining particular rises or falls would require corporate announcements, market-index movements, and macroeconomic information.

C. Trading Volume

Fig. 2 shows daily trading volume. The mean is approximately 4,941,709 shares (units as reported in the source file), whereas the median is 3,427,885; this gap indicates a right-skewed distribution. A small number of very active sessions, the largest reaching 201,129,980, dominate the vertical scale and pull the mean upward.

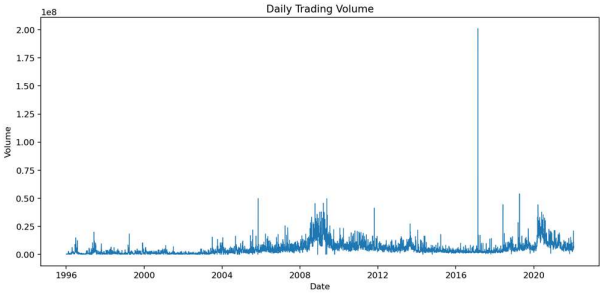


Fig. 2. Daily trading volume across the dataset.

Volume is useful context because a price movement accompanied by unusually high activity may differ from one during quiet trading. However, the file does not identify investor categories, order direction, or the reason for a transaction, so volume is interpreted only as an activity measure. Percentiles, rolling averages, year-wise summaries, or a logarithmic axis would allow a deeper study of this skewed series.

D. Daily Returns and Volatility

Price levels trend over time and are not directly comparable across periods, whereas percentage returns express each change relative to the previous observation. The daily return is

$$R_t = (P_t / P_{t-1}) - 1 \qquad (1)$$

where P_t and P_{t-1} are the adjusted closes of the current and previous available observations. The mean daily return is 0.12141% and the standard deviation is 2.15914%. These values describe the sample period and are not estimates of future return or risk. Fig. 3 shows the histogram of returns, which is clipped at the 1st and 99th percentiles for display only; the reported statistics use the unclipped series.

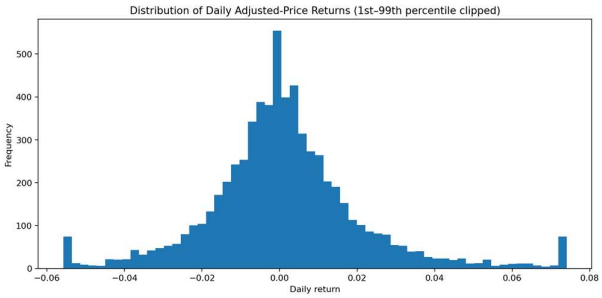


Fig. 3. Distribution of daily adjusted-price returns (tails clipped at the 1st and 99th percentiles for display).

The returns are concentrated near zero with tails on both sides. Financial returns often contain outliers and may not be normally distributed, so the histogram is a descriptive tool and not a complete model of risk. The standard deviation does not capture drawdown, liquidity risk, event risk, or changing volatility regimes; rolling volatility, maximum drawdown, and downside deviation would extend the analysis. The return histogram complements the price chart: the first shows the path of the asset's value, the second the size and frequency of relative changes.

E. Correlation Analysis

The Pearson correlation coefficient between variables X and Y is

$$r = \mathrm{cov}(X, Y) / (\sigma_X \sigma_Y) \quad (2)$$

and ranges from −1 to +1. As Fig. 4 shows, Open, High, Low, Close, and Adj Close are almost perfectly correlated ($r \approx 1.00$), which is expected because they describe the same trading session and implies redundancy among the raw price inputs. Volume has a weak positive correlation of about 0.24 with the price fields.

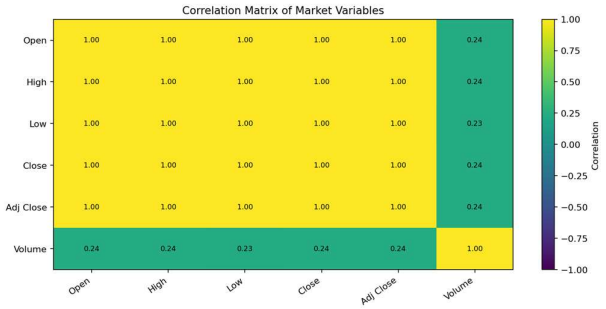


Fig. 4. Correlation matrix of the numeric market variables.

Correlation measures linear association only: a low value does not prove independence, because a nonlinear relationship may exist, and a high value does not mean that one variable causes another. The heat map therefore guides further investigation rather than serving as a causal explanation, and the redundancy among price fields motivates the feature-engineering stage, where lagged values and rolling summaries represent recent market history.

F. Year-End Prices

The daily adjusted close was resampled to year-end frequency by retaining the last available observation in each year, which is not necessarily 31 December because the exchange may not have traded on that date. Fig. 5 shows year-end values rising steadily over the sample, with larger annual increases in later years. Because the data end on 21 January 2022, the final point is not a full-year value.

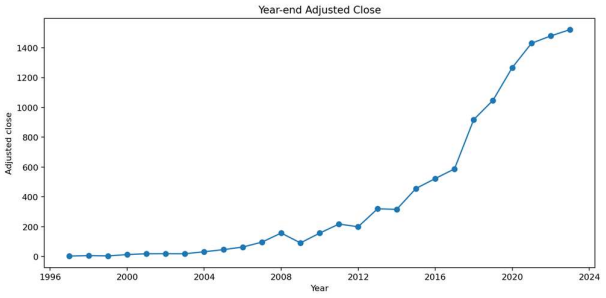


Fig. 5. Year-end adjusted closing price.

Annual aggregation is effective for communicating long-term price levels but removes intra-year information: two years may have similar year-end values despite very different paths. Fig. 5 therefore complements rather than replaces Fig. 1. Annual return, annualized volatility, and maximum drawdown are natural extensions.

IV. Feature Engineering and Leakage Prevention

A. Engineered Features

Feature engineering transforms raw columns into variables that may better represent the structure of the prediction problem. In a time-series setting the features must be calculated using only information available at or before the prediction date. In addition to Open, High, Low, Close, and Volume, the engineered inputs in Table IV were used. Rolling features require prior history, so the first rows contain missing values, and the final row has no next-row target. After removing such rows the modeling table contains 6,540 records.

TABLE IV. Engineered Features

Feature	Definition
Return_1d, Return_5d	One- and five-observation percentage change
MA_5, MA_10, MA_20	Rolling means of adjusted close
Volatility_10d	Rolling standard deviation of daily returns
Range_pct	(High − Low) / Close
Day_of_week, Month, Year	Calendar fields derived from Date
AdjClose_lag_1/2/3/5	Prior adjusted closing prices
Target	Adjusted close of the next row

B. Prevention of Time-Series Leakage

Leakage occurs when a model is trained with information that would not have been available at prediction time. Here the target is shifted by one row, lag and rolling features use only current or past values, and the data are split chronologically: the earlier 80% (5,232 rows) is used for training and the later 20% (1,308 rows) for testing. Median imputation is placed inside a scikit-learn Pipeline [5], so the imputation values are learned from the training portion only and then applied to the test portion. Fitting preprocessing on the entire dataset before splitting could allow information from the test period to influence training.

V. Machine Learning Models and Evaluation

Five regression models [5] receive the same engineered features and are evaluated on the same chronological test set, which makes the comparison more meaningful than evaluating each model on a different subset of observations.

A. Regression Models

Linear Regression models the target as a weighted sum of the inputs plus an intercept. It is a simple, interpretable baseline that can perform well when the relationship is approximately linear, but it may be sensitive to multicollinearity, outliers, and nonlinear patterns.

Ridge Regression adds an L2 penalty to the linear objective, discouraging excessively large coefficients. Regularization can help when inputs are correlated or a model risks overfitting; the parameter alpha sets the penalty strength.

Random Forest is an ensemble of decision trees trained on randomized samples and feature subsets, with their predictions combined. It can model nonlinear relationships and interactions without a linear functional form, and depends on settings such as the number of trees, maximum depth, and minimum leaf size.

Gradient Boosting builds an additive sequence of weak learners, commonly shallow trees, each reducing the errors of the current ensemble. The learning rate, number of estimators, and tree depth balance fit and generalization, and boosting may overfit if complexity is not controlled.

Extra Trees (extremely randomized trees) is an ensemble of randomized trees that adds further randomness when selecting split thresholds. This can reduce variance in some settings, although performance depends on the data and hyperparameters.

B. Training Procedure

A Pipeline is created for each algorithm: median imputation first, then the estimator. Each model is trained on the first 80% of the chronological modeling table and predicts the remaining 20%. The same test targets are used to compute all three metrics, and the model with the lowest RMSE is selected.

C. Evaluation Metrics

With actual values y_i, predictions $\hat{y}_i$, and n test observations, the mean absolute error (MAE), root mean squared error (RMSE), and coefficient of determination (R^2) are

$$\text{MAE} = (1/n)\ \Sigma\ |y_i - \hat{y}_i| \tag{3}$$

$$\text{RMSE} = \sqrt{[(1/n)\ \Sigma\ (y_i - \hat{y}_i)^2]} \tag{4}$$

$$R^2 = 1 - \Sigma\ (y_i - \hat{y}_i)^2\ /\ \Sigma\ (y_i - \bar{y})^2 \tag{5}$$

where $\bar{y}$ is the mean of the actual test values. MAE and RMSE are in adjusted-price units, and lower values mean smaller error; RMSE penalizes large errors more heavily than MAE. A value of R^2 closer to 1 means the model explains a larger share of the variation in the test target, while a negative value means the model performs worse than a constant mean predictor on that test set. R^2 should not be interpreted alone.

D. Results

Table V lists the test performance. Linear Regression has the lowest RMSE and is therefore selected, with an MAE of 13.1117, an RMSE of 19.5296, and an R^2 of 0.9949. Ridge Regression is nearly identical, whereas the three tree ensembles have RMSE values above 538 and negative R^2, as compared in Fig. 6. Because RMSE is in adjusted-price units, it is directly comparable across the models on the same test set.

TABLE V. MODEL PERFORMANCE ON THE CHRONOLOGICAL TEST SET

Model	MAE	RMSE	R^2
Linear Regression	**13.1117**	**19.5296**	**0.9949**
Ridge Regression	13.1314	19.5327	0.9949
Extra Trees	466.4015	538.8825	−2.8519
Random Forest	469.3221	541.6142	−2.8910
Gradient Boosting	475.6073	547.2583	−2.9725

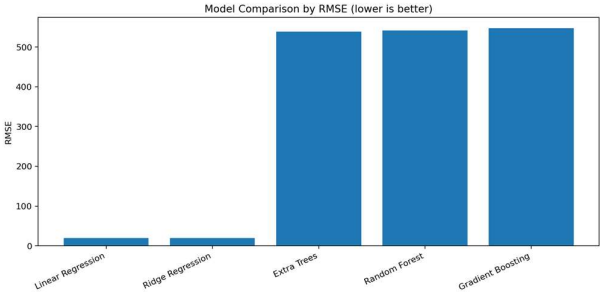


Fig. 6. Model comparison by test RMSE (lower is better).

The substantial gap between the linear and tree-ensemble models in this run should prompt further investigation rather than an assumption that one model family is always superior. Price-level targets are strongly related to recent price levels, so the evaluation can reward a model for tracking the prevailing level and a high R^2 may partly reflect this. A naïve persistence baseline, which predicts the current adjusted close as the next one, was not evaluated and would be needed to show whether the fitted models add value. One possible factor in the weak tree results is that tree ensembles cannot extrapolate beyond the target range seen in training; this was not tested here.

Fig. 7 shows that the selected model follows the broad movement of the held-out target series. Agreement in the general level does not by itself imply accurate prediction of daily changes. Plotting prediction residuals and comparing predicted returns with actual returns would be a useful extension.

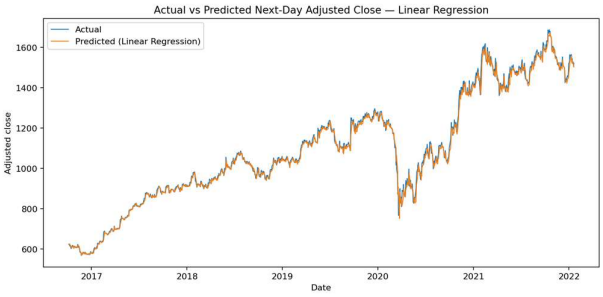


Fig. 7. Actual and predicted next-observation adjusted close for the selected model (Linear Regression).

VI. CONCLUSION AND FUTURE SCOPE

A. Conclusion

This paper completed an end-to-end data science workflow on historical HDFC Bank market data: dataset inspection, date parsing, numeric conversion, duplicate and invalid-value checks, missing-value treatment, five exploratory analyses, feature engineering, chronological splitting, regression training, and evaluation. The EDA covered the adjusted-price trend, daily trading volume, daily returns and volatility, correlation among market variables, and year-end prices, which gave different views of the same data before machine learning began.

Under the minimum-RMSE criterion, Linear Regression performed best in the recorded run (MAE 13.1117, RMSE 19.5296, R^2 0.9949). The main lesson is that model selection must rest on a clearly defined evaluation design: a high R^2 on a price-level target does not show that a model can forecast market direction or support profitable trading. The work is an

academic demonstration of data science and machine learning methods.

B. Limitations

The data end on 21 January 2022, and the provider, download date, and adjustment conventions of the adjusted-close series must be confirmed; the fields were used as supplied and were not validated against an exchange feed. The target is the next available row, not always the next calendar day. The tree models performed poorly in the recorded run, but this may change with a different period, feature set, parameters, or validation method. No transaction costs, slippage, trading strategy, or risk management were modeled, so the results cannot be translated into a claim about investment profitability.

C. Future Scope

- Refresh the dataset from a verified source, record its URL, provider, ticker, download date, and licensing, and rerun the notebook.
- Add a naïve persistence baseline and use walk-forward or expanding-window validation across multiple test periods.
- Predict returns or direction as well as price levels, and evaluate directional accuracy.
- Tune hyperparameters with time-series-aware validation instead of random cross-validation.
- Add residual analysis, error distributions, and error comparisons across market regimes.
- Explore benchmark and sector index returns, company fundamentals, and macroeconomic indicators, and study feature importance while accounting for correlated predictors.

Any extension must preserve chronological integrity: every added feature must be available at prediction time, and external information should be timestamped and aligned with the market observation date to avoid leakage.

ACKNOWLEDGMENT

The author thanks Lovely Professional University and the project supervisor, [maneet kaur], for guidance and feedback, the faculty of the Computer Science and Engineering discipline, and the developers and maintainers of Python, pandas, NumPy, Matplotlib, and scikit-learn.

REFERENCES

[1] W. McKinney, *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.

[2] "pandas documentation." [Online]. Available: https://pandas.pydata.org/docs.

[3] "NumPy documentation." [Online]. Available: https://numpy.org/doc.

[4] "Matplotlib documentation." [Online]. Available: https://matplotlib.org/stable.

[5] "scikit-learn documentation." [Online]. Available: https://scikit-learn.org/stable .

[6] "HDFCBANK.NS.csv," historical market data set supplied for this project. [Online].